M10 - Docker Airflow

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

This is the primary teacher.

Do not use the quick reference as the main lesson. Read the concept, run/inspect the demonstration, prove the result, then practise independently.

Contents

DE10-01 - Why containers exist

What you will be able to do

Explain the problem containers solve and distinguish application reproducibility from virtual machines/production orchestration.

Why this matters in real Data Engineering

"Works on my machine" often means hidden OS/package/runtime differences. Containers package an application filesystem/runtime/config boundary so the same image can run consistently.

Picture it this way

A container image is a sealed travel kit containing the app and runtime it needs; the host still provides the building/kernel.

Start from zero

- A container is an isolated process/filesystem/network namespace environment built from an image; it is not a full separate hardware machine.
- An **image** is an immutable template/layer stack; a **container** is a running/stopped instance of that image.
- Containers improve reproducibility and dependency isolation, but external data/secrets/persistent state still need explicit handling.
- Docker Desktop/Engine is runtime tooling; verify it works before reinstalling.
- A container can be ephemeral: deleting it may delete its writable layer. Persistent data belongs in volumes/bind mounts/external services.
- Do not put credentials inside an image because anyone with image access/layer history may recover them.

Teacher demonstration

```
# Verify before changing setup
docker version
docker info
docker run --rm hello-world
```

Read the example like English

- docker version proves client/server reachability.
- hello-world is a disposable smoke test; --rm removes stopped container automatically.
- If Docker command fails, diagnose engine/context before changing application code.

Expected check

concept	meaning
image	immutable build artifact/template
container	runtime instance/process
host	provides Docker engine/kernel/resources
volume/external DB	durable state outside container writable layer

Common beginner traps

- Reinstalling Docker because a compose project fails.
- Calling a container a lightweight VM without noting shared host kernel model.
- Storing database files only in a disposable container layer.

Now you do a similar task

Verify Docker with version/info/hello-world if available. Write a short "works on my machine" example that a container image would remove and one problem it would NOT remove.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If an image is reproducible, why can two container runs still behave differently?

Answer in your own words before continuing.

DE10-02 - Images and containers

What you will be able to do

Inspect images/containers and understand create/start/run/stop/remove lifecycle without confusing image deletion with container deletion.

Why this matters in real Data Engineering

Docker debugging becomes much easier when you know whether the problem is build-time image content or runtime container state.

Picture it this way

An image is a cookie cutter; containers are cookies made from it. Deleting one cookie does not delete the cutter.

Start from zero

- docker image ls shows local images; docker ps shows running containers; docker ps -a includes stopped.
- docker run creates + starts a new container; docker start restarts an existing stopped container.
- Container writable changes are instance-local unless stored through volume/bind/external service.
- docker inspect reveals configuration such as mounts/network/ports/environment (be careful with secrets).
- Logs belong to a container/process; image build output is separate evidence.
- Use names/labels in training to know exactly which resource you are touching.

Teacher demonstration

```
docker image ls
docker ps -a
# Training example
docker run --name m10-demo python:3.13-slim python -c "print('hello from container')"
docker ps -a --filter name=m10-demo
docker logs m10-demo
docker rm m10-demo
```

Read the example like English

- run pulls image if needed, creates container and executes Python command.
- After the command exits, container is stopped but still exists because --rm was not used.
- logs reads stdout/stderr from that container.
- rm removes container, not the python image.

Expected check

command	effect
docker run	create + start new container
docker start	start existing stopped container
docker rm	remove container
docker rmi	remove image if not required

Common beginner traps

- Deleting images/containers randomly before reading logs.
- Assuming a stopped container has disappeared.
- Using docker prune as first troubleshooting step.

Now you do a similar task

Create one harmless stopped demo container, inspect logs/state, remove it, and prove the base image still exists or can be pulled again.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What is preserved when a container stops, and what disappears only when the container is removed?

Answer in your own words before continuing.

DE10-03 - Writing a Dockerfile

What you will be able to do

Read/write a Dockerfile as a reproducible build recipe with deliberate base image, workdir, dependency caching, non-root runtime and command.

Why this matters in real Data Engineering

A Dockerfile is part of the production artifact. Small mistakes can leak secrets, bloat images or make builds non-reproducible.

Picture it this way

The Dockerfile is the packing checklist used to build the sealed travel kit. Order matters because layers can be reused.

Start from zero

- FROM selects a base image; pin/choose deliberately.
- WORKDIR sets default path for following instructions/runtime.
- COPY dependency manifest before source often improves build cache reuse.
- RUN executes at build time and creates image layer changes.
- CMD/ENTRYPOINT define default runtime process.
- Use .dockerignore to avoid copying .git, .venv, secrets and unnecessary data.
- Prefer non-root application user where practical; root inside container still has elevated power within the container namespace.

Teacher demonstration

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
RUN useradd --create-home appuser
USER appuser
CMD ["python", "src/pipeline.py"]
```

Read the example like English

- This is the supplied M10 application Dockerfile.
- Dependencies are installed before copying source so normal source edits can reuse the pip layer.
- USER appuser means runtime command is not root.
- No secret file is copied into the image.

Expected check

check	evidence
build context	.dockerignore reviewed
base	known image/tag
secrets in Dockerfile/image	none
runtime user	non-root appuser
command	pipeline.py

Common beginner traps

- COPY . . without .dockerignore.
- ARG/ENV hardcoded secret in image.
- Using latest everywhere and never recording tested versions.

Now you do a similar task

Review the supplied Dockerfile line-by-line. Build it if Docker works, then inspect image history/config and prove no training secret file was copied.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can copying requirements.txt before source code make rebuilds faster?

Answer in your own words before continuing.

DE10-04 - Ports

What you will be able to do

Distinguish container ports, host published ports and service-to-service communication.

Why this matters in real Data Engineering

Port confusion is one of the most common beginner Docker bugs: localhost means different things depending on which process/container you are in.

Picture it this way

A container is an apartment. The app listens on a room number inside; port publishing maps a building entrance number on the host to that internal room.

Start from zero

- A process must listen on a container port; Docker port publishing maps host_port:container_port.
- Inside one container, localhost refers to that same container, not the host or another service.
- Compose services normally reach each other by service name + container port over a Compose network.
- Publishing a database port to the host is only needed for host access; container-to-container traffic does not need the host mapping.
- 0.0.0.0 vs 127.0.0.1 bind address affects reachability of the process.
- Use docker compose ps / docker port / inspect and application logs to diagnose.

Teacher demonstration

```
# Example syntax
ports:
  - "8080:8080"  # host 8080 -> container 8080
# Inside pipeline container for Compose Postgres:
DB_HOST=db
DB_PORT=5432
# NOT localhost:5432
```

Read the example like English

- The Airflow training compose publishes web UI 8080:8080 for your browser.
- The pipeline service reaches Postgres at hostname db because db is the Compose service name.
- No host port mapping is required for that internal database connection.

Expected check

caller	correct destination
host browser	localhost:published_host_port
pipeline container	db:5432
db container itself	localhost:5432 if server listens there

Common beginner traps

- Using localhost for another Compose service.
- Changing random ports because DB auth failed.
- Publishing every internal service port unnecessarily.

Now you do a similar task

Inspect compose.yaml and write the exact address used by (a) browser to Airflow and (b) pipeline container to Postgres. If Docker works, verify with compose config/ps.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why does localhost inside the pipeline container not mean the Postgres service?

Answer in your own words before continuing.

DE10-05 - Volumes and persistence

What you will be able to do

Choose bind mounts, named volumes or ephemeral container storage based on whether data/config/code must persist/share.

Why this matters in real Data Engineering

If a database container is recreated without a volume, its writable-layer data may vanish. If source code is baked/mounted incorrectly, runtime may not see expected files.

Picture it this way

Container writable storage is a hotel room; a named volume is a storage locker; a bind mount is a door into a specific host folder.

Start from zero

- Container writable layer is tied to the container lifecycle.
- Named volumes are managed by Docker and survive container replacement until explicitly removed.
- Bind mounts map a host path into a container path and are useful for local code/data sharing.
- The supplied Postgres uses named volume db_data for durable database files.
- The supplied pipeline bind-mounts ./data:/app/data so host-visible input/output is shared.
- Mounts can hide files that existed at the image path; inspect destination carefully.
- Backups still matter: a volume is persistence, not automatically a backup.

Teacher demonstration

```
    volumes:
      - db_data:/var/lib/postgresql/data
    # Pipeline
    volumes:
      - ./data:/app/data
```

Read the example like English

- db_data survives replacing the db container.
- The data bind mount makes orders.csv/output accessible to both host and pipeline path /app/data.
- Removing a named volume is a destructive operation; do not use compose down -v casually.

Expected check

storage	lifecycle/use
container layer	ephemeral instance state
named volume	Docker-managed persistent data
bind mount	specific host directory shared into container

Common beginner traps

- docker compose down -v without knowing it deletes volumes.
- Calling a volume a backup.
- Mounting an empty host folder over a path containing required image files.

Now you do a similar task

Map every mount in the supplied Compose file: source, destination, owner/persistence intent. Explain what would be lost by down -v.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can a bind mount make files that were built into an image appear to disappear at runtime?

Answer in your own words before continuing.

DE10-06 - Networks and service communication

What you will be able to do

Understand Compose networking, DNS service names and health/dependency readiness instead of using host assumptions.

Why this matters in real Data Engineering

Multiple services may start at different times. A container being "started" does not mean the database is ready to accept connections.

Picture it this way

Compose is a small private office network: each service gets a name/address; a receptionist healthcheck can signal when a dependency is actually ready.

Start from zero

- Compose creates a default network where services resolve each other by service name.
- Use db:5432 for the Postgres service from another Compose service.
- depends_on controls startup relationships; with health condition it can wait for a declared healthcheck in supported Compose behavior.
- A healthcheck is an explicit command proving service readiness, not just process existence.
- Applications still need retry/failure handling because dependencies can become unavailable after startup.
- Network bugs are diagnosed with compose config/ps/logs, service names/ports and health status.

Teacher demonstration

```
    db:
      healthcheck:
        test: ["CMD-SHELL", "pg_isready -U m10_user -d m10"]
        interval: 5s
        timeout: 5s
        retries: 10
    pipeline:
      depends_on:
        db:
          condition: service_healthy
```

Read the example like English

- Postgres healthcheck asks pg_isready whether the database accepts connections.
- Pipeline startup is delayed until db is healthy in this lab Compose.
- This does not guarantee db will remain healthy forever.

Expected check

state	meaning
container running	process exists
healthy	healthcheck currently passes
service reachable	network/DNS/port + app readiness
application success	its own work completed

Common beginner traps

- Using depends_on as a permanent reliability guarantee.
- Trying host machine IP before using Compose service DNS.
- Changing app DB_HOST to localhost.

Now you do a similar task

Run offline compose validator, and if Docker is available run compose config/up. Inspect db health and pipeline logs. Explain why healthcheck is stronger than start order.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What failure can still happen even if depends_on initially waited for a healthy database?

Answer in your own words before continuing.

DE10-07 - Compose, configuration and secrets

What you will be able to do

Use Compose to declare multiple services/configuration while keeping secrets out of images and committed plaintext.

Why this matters in real Data Engineering

Container reproducibility depends on explicit configuration. Secret handling is part of that contract, not an afterthought.

Picture it this way

Compose is the site plan listing buildings, wiring, storage and private keys; secrets should be delivered at runtime, not painted on the blueprint.

Start from zero

- Compose describes services, build/image, environment, networks, volumes, secrets, healthchecks and dependencies.
- Run **docker compose config** before starting to catch interpolation/schema problems.
- Environment variables are configuration; sensitive values should use secret mechanisms rather than committed literals.
- The supplied Compose uses POSTGRES_PASSWORD_FILE and Docker Compose secret mounted under /run/secrets.
- The committed file is db_password.txt.example only; the real db_password.txt must remain untracked.
- Do not paste compose config output publicly if it contains expanded sensitive environment values in other projects.

Teacher demonstration

```
secrets:
  db_password:
    file: ./secrets/db_password.txt
services:
  db:
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
```

Read the example like English

- The secret content is not written in compose.yaml.
- Runtime reads the mounted secret file.
- The example file documents setup without containing a real credential.

Expected check

artifact	safe?
compose.yaml	tracked config, no secret value
db_password.txt.example	tracked placeholder
db_password.txt	local secret, ignored
image layers	no password value

Common beginner traps

- Committing a real .env/password file.
- Putting secret in Dockerfile ENV.
- Assuming a Docker secret makes an already-committed credential safe.

Now you do a similar task

Create the local fake training password file from the .example (use a fake value), run git status/check-ignore, then compose config. Prove the real local secret file is not tracked.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If a password was already committed to Git, why is moving it to a Compose secret not enough?

Answer in your own words before continuing.

DE10-08 - Why orchestration exists

What you will be able to do

Explain what workflow orchestration adds beyond a cron job or one monolithic Python script.

Why this matters in real Data Engineering

Production data jobs have dependencies, schedules, retries, backfills, logs and state across many tasks. Orchestrators coordinate these responsibilities.

Picture it this way

An orchestrator is an airport control tower: it does not fly each plane, but coordinates when independent tasks may start, retries/failures and operational visibility.

Start from zero

- Orchestration defines task dependencies and when logical data intervals should be processed.
- Airflow is primarily a workflow orchestrator/scheduler, not a distributed data-processing engine.
- Tasks should call bounded work in external systems/code; avoid passing huge datasets through orchestration metadata.
- A DAG describes dependency graph; a DAG run is one logical execution; task instances are executions of tasks within runs.
- Retries/backfills require idempotent task behavior or durable key/window design.
- UI/logs give operational visibility but do not replace downstream data validation.

Teacher demonstration

```
extract >> validate >> transform >> load
# Airflow coordinates these task boundaries.
# The heavy transformation could still run in Python/SQL/Spark elsewhere.
```

Read the example like English

- Dependency arrows are about ordering/requirements, not data volume transfer.
- If validate fails, downstream transform/load should not run for that DAG run unless trigger rules explicitly say otherwise.
- A scheduler gives repeatable run identity and operational history.

Expected check

tool	best fit
cron	simple independent command schedule
Airflow	dependency-rich batch workflows/backfills/operations
Spark	distributed data processing
Docker	runtime packaging

Common beginner traps

- Calling Airflow a database/ETL engine.
- One giant task containing the entire pipeline so dependencies/retry scope disappear.
- Using orchestration without idempotent data writes.

Now you do a similar task

Take a four-step batch pipeline and explain what Airflow would coordinate versus what the Python/SQL jobs themselves still must guarantee.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can Airflow retry a task correctly but the data still duplicate?

Answer in your own words before continuing.

DE10-09 - Airflow architecture and UI

What you will be able to do

Identify Airflow components/UI objects and diagnose a specific failed DAG run/task instance instead of "restarting Airflow" blindly.

Why this matters in real Data Engineering

Operational debugging depends on knowing whether the DAG parsed, scheduler created a run, task was queued/running/failed and what its log says.

Picture it this way

Airflow UI is a control room showing workflow plans, runs and individual task states; the first job is to locate the exact red task, not reboot the building.

Start from zero

- Airflow 3 training image in this course uses apache/airflow:3.3.0 and TaskFlow imports from airflow.sdk.
- The local standalone command is a training convenience that starts required components for one-machine learning.
- DAG list/grid/graph views show parsed workflows, runs and task dependencies/state.
- A failed task instance has logs and try number; read that before clearing/retrying.
- DAG parse/import errors are different from runtime task errors.
- Run state is orchestration evidence, not proof output data is reconciled.

Teacher demonstration

```
# Training compose
services:
  airflow:
    image: apache/airflow:3.3.0
    command: standalone
    ports:
      - "8080:8080"
    volumes:
      - ./dags:/opt/airflow/dags
      - ../data:/opt/airflow/data
```

Read the example like English

- Browser reaches local Airflow at localhost:8080 after service is ready.
- DAG files are bind-mounted so edits are visible to the training service.
- Data folder is separately mounted for the DAG tasks.

Expected check

debug layer	evidence
parse	DAG appears/no import error
schedule/run	DAG run exists
task	specific task instance state
cause	task log traceback/message

Common beginner traps

- Clearing every task before reading the log.
- Treating DAG not visible and task failed as same problem.
- Assuming green tasks prove business output.

Now you do a similar task

Run the offline DAG validator. If Docker is available, start the Airflow compose and identify the daily_orders_training DAG, a run and task logs. Save the exact component/state evidence.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What is the first evidence you need to distinguish a DAG import problem from a task runtime failure?

Answer in your own words before continuing.

DE10-10 - DAGs, tasks and dependencies

What you will be able to do

Design a DAG with tasks that correspond to meaningful retry/validation boundaries and explicit dependencies.

Why this matters in real Data Engineering

Task boundaries decide what can retry independently and what operational state Airflow exposes.

Picture it this way

Divide a factory line into stations where each station can fail/retry without repeating unrelated work.

Start from zero

- A DAG must be acyclic; dependencies form a directed acyclic graph.
- Task should have a clear responsibility/input/output and idempotent durable effects if retried.
- Separate validation from load so bad data blocks publication.
- Avoid thousands of tiny tasks for row-level work and one giant task for an entire complex pipeline; choose operationally meaningful boundaries.
- Dependencies should reflect data/control prerequisites, not merely visual order.
- Use task IDs that state action/domain and keep DAG code versioned/testable.

Teacher demonstration

```
@dag(...)
def daily_orders_training():
    a = extract()
    b = validate(a)
    c = transform(b)
    load(c)
daily_orders_training()
```

Read the example like English

- The functional TaskFlow calls create dependencies through returned task outputs.
- extract -> validate -> transform -> load matches the supplied training pipeline boundaries.
- If validate raises, downstream tasks are not normal successful candidates.

Expected check

task	responsibility
extract	identify source/window + metadata
validate	contract/reconciliation prerequisites
transform	deterministic business shape
load	idempotent durable publication

Common beginner traps

- One task per source row.
- Load embedded inside transform with no validation gate.
- Dependencies that do not match data prerequisites.

Now you do a similar task

Draw a DAG for source file -> schema check -> transform -> target load -> reconciliation. Explain which task you would retry after a temporary target DB failure.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why might putting extraction and database load in the same task make retry behavior riskier?

Answer in your own words before continuing.

DE10-11 - TaskFlow, operators and task communication

What you will be able to do

Use TaskFlow/operator concepts and small metadata communication without passing large dataframes through Airflow metadata.

Why this matters in real Data Engineering

Airflow tasks need to communicate control metadata, but the orchestration database is not meant to become your data lake.

Picture it this way

Pass a shipping receipt between factory stations, not the entire truckload through the manager's notebook.

Start from zero

- TaskFlow @task decorates Python functions as tasks and captures dependencies/return metadata.
- Traditional operators are task classes for common actions; TaskFlow and operators can coexist.
- Small return values are stored/transported through Airflow metadata/XCom mechanisms.
- Pass paths, table names, row counts, run/window IDs - not huge datasets.
- Durable datasets belong in files/tables/object storage; downstream tasks receive their location/identity.
- Task communication must not expose secrets in metadata/UI.

Teacher demonstration

```
@task
def extract():
    return {"source_path": "/opt/airflow/data/orders.csv", "source_rows": 4}
@task
def validate(meta):
    if meta["source_rows"] <= 0:
        raise ValueError("empty source")
    return {**meta, "validation": "PASS"}
```

Read the example like English

- The returned dictionary is tiny control metadata.
- The CSV itself remains in the mounted data path.
- Downstream receives metadata and can open durable data by path if needed.

Expected check

good task metadata	bad task metadata
path/table/partition ID	full million-row DataFrame
row counts/status	secret token/password
run/window identity	large binary payload

Common beginner traps

- Returning entire pandas dataframe through XCom.
- Putting database password in task return/log.
- Treating TaskFlow return value as durable data storage.

Now you do a similar task

Modify a toy DAG metadata flow to return only a path + counts between tasks. Explain where the actual dataset lives.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is a file/table path usually a better task-to-task message than the full dataset?

Answer in your own words before continuing.

DE10-12 - Scheduling and data intervals

What you will be able to do

Reason about schedules using logical data intervals instead of assuming run timestamp equals the data window being processed.

Why this matters in real Data Engineering

Daily pipelines often process a completed prior interval. Misunderstanding schedule semantics causes off-by-one-day loads and unsafe backfills.

Picture it this way

A newspaper printed at 1 AM may summarize yesterday; the print/run time is not necessarily the reporting period.

Start from zero

- A DAG schedule defines when runs are created for logical intervals; do not hardcode today() as business window.
- Use Airflow-provided logical data interval/context or explicit parameters so rerun/backfill processes the intended historical window.
- start_date establishes scheduling boundary; it is not "run immediately at this timestamp" in the naive sense.
- catchup controls whether scheduler creates historical interval runs from start date under schedule semantics.
- Timezone should be explicit/consistent with source/business requirements.
- Tasks should produce paths/queries keyed by logical interval so each run is idempotent.

Teacher demonstration

```
@dag(
    dag_id="daily_orders_training",
    schedule="@daily",
    start_date=datetime(2026, 8, 1),
    catchup=False,
)
def daily_orders_training():
    ...
```

Read the example like English

- The supplied DAG is deliberately simple and catchup=False for first learning.
- A production task should use the run/data interval rather than datetime.now() to choose historical source date.
- This makes a manual retry/backfill reproduce the original logical day.

Expected check

concept	question
run time	when scheduler/task executes
data interval	which logical period run represents
start_date	schedule boundary
catchup	whether missed/historical intervals are scheduled

Common beginner traps

- Using current date inside task for a historical rerun.
- Thinking start_date means first task instantly executes.
- Backfilling with a script that writes to today partition.

Now you do a similar task

Write a daily partition path pattern based on logical interval, not wall-clock now. Explain what partition a retry of an old run should write.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is datetime.now() dangerous inside a task that may be backfilled later?

Answer in your own words before continuing.

DE10-13 - Retries, timeouts and failure behavior

What you will be able to do

Configure retries/timeouts according to failure class and prove a retry cannot duplicate durable data.

Why this matters in real Data Engineering

Airflow can retry automatically, but orchestration retries are safe only when task side effects are designed for replay.

Picture it this way

A failed card payment retry is safe only if the merchant has an idempotency key; otherwise the automation may charge twice.

Start from zero

- Retries are appropriate for transient errors; deterministic code/schema/credential errors should fail fast for repair.
- retry_delay/backoff control timing; do not use huge retry counts to hide instability.
- Execution/task timeout bounds hanging work.
- A retried load task should upsert/replace a known partition/use run key rather than blind append.
- Log attempt/try number and failure classification.
- After retries exhausted, downstream publish/state should remain safe and alert/run status should show failure.

Teacher demonstration

```
@task(retries=2, retry_delay=timedelta(seconds=30))
def extract():
    ...
# Durable load idea:
# MERGE/UPSERT by business key OR replace the exact data_interval partition.
```

Read the example like English

- The supplied extract task allows two retries after the first attempt.
- The task itself must still be safe to execute again.
- A bad source schema would usually not be repaired by waiting 30 seconds.

Expected check

failure	retry?
temporary network/503	yes, bounded
rate limit	yes, policy-aware
invalid credentials	usually no identical retry
schema contract break	no; repair/quarantine
load after commit response lost	retry only with idempotent target

Common beginner traps

- Adding retries to every task.
- Blind append load on retry.
- Clearing failed task repeatedly without reading log/cause.

Now you do a similar task

Classify five failure examples for the training DAG and write an idempotency strategy for load. If Docker works, trigger a controlled failure and inspect try/log behavior.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What failure is especially dangerous when the database commit succeeded but the task lost the success response?

Answer in your own words before continuing.

DE10-14 - Parameters and task communication

What you will be able to do

Use DAG/task parameters and runtime metadata for small controlled variation without changing source code for every run.

Why this matters in real Data Engineering

Operational workflows need dates, file paths, backfill modes and thresholds, but ad-hoc code edits destroy reproducibility.

Picture it this way

Parameters are fields on the job order; task communication is the receipt passed between stations. Neither should become a secret store or giant data transport.

Start from zero

- DAG parameters/config can provide controlled user/run inputs.
- Validate parameters: type, allowed range/path/date and whether they change idempotency key/output partition.
- Use logical interval context for scheduled values; manual parameters should override only where contract allows.
- Pass small metadata through task outputs/XCom, durable data by reference.
- Record parameter values in run lineage so a manual run is reproducible.
- Never accept arbitrary SQL/path commands from an untrusted parameter without validation.

Teacher demonstration

```
# Conceptual manual run config
{
  "source_date": "2026-08-15",
  "mode": "backfill"
}
# Validate date/mode, then derive the exact input/output partition.
```

Read the example like English

- Run config makes a manual backfill explicit.
- The derived partition belongs to source_date, not current wall clock.
- Lineage should record these non-secret parameters.

Expected check

parameter	validation
source_date	ISO date + permitted range
mode	allowed enum normal/backfill
threshold	numeric bounded
secret	not a normal DAG param

Common beginner traps

- Editing DAG code for each backfill date.
- Allowing arbitrary filesystem path/SQL from UI param.
- Passing tokens through XCom/run config.

Now you do a similar task

Design three safe parameters for a historical pipeline run and show how they affect input/output identity. State validation rules.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why should a backfill date become part of output/run identity?

Answer in your own words before continuing.

DE10-15 - Connections, variables and secrets

What you will be able to do

Separate Airflow connections/variables/config from secrets and choose the least exposed storage mechanism for credentials.

Why this matters in real Data Engineering

DAG code is versioned/shared; credentials rotate and should not be pasted into repositories or logs.

Picture it this way

The workflow blueprint can name which locked key is needed; it should not print the key value on the blueprint.

Start from zero

- Airflow Connections model external system connection metadata/credentials; Variables hold general runtime configuration and are not a reason to store every secret.
- Environment/secret backends/external secret managers can supply sensitive values depending deployment.
- DAG code should refer to connection IDs/config names rather than embedded passwords.
- UI/log/XCom/rendered templates can expose values; avoid logging secret objects/connection URIs.
- Use separate dev/test/prod credentials with least privilege.
- Rotate a leaked credential; deleting it from the latest file is not sufficient.

Teacher demonstration

```
# Better DAG idea
connection_id = "orders_postgres"
# Task/operator/hook resolves credentials at runtime.
# Do NOT write: password="MyRealPassword" in DAG code.
```

Read the example like English

- The ID is safe to commit; the credential value is environment-managed.
- Different environments can map the same logical ID to different credentials.
- Airflow metadata security still matters because authorized users may inspect connection configuration.

Expected check

item	where
connection ID	versioned DAG/config
password/token	connection/secret backend/runtime
non-secret threshold	variable/config/param
secret in log/XCom	never intentionally

Common beginner traps

- Putting secrets in DAG Python.
- Using Variables as plaintext dumping ground because they are convenient.
- Same owner/superuser database credential for every DAG/environment.

Now you do a similar task

Audit the training project for password/token paths. Explain which files are committed examples, which local runtime secrets are ignored, and how an Airflow connection ID would replace hardcoding.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What is the benefit of keeping the same connection ID while changing the actual credential between dev and prod?

Answer in your own words before continuing.

DE10-16 - Sensors and waiting

What you will be able to do

Use sensors/waiting only when a workflow must wait for an external condition, with bounded timeout and resource-aware behavior.

Why this matters in real Data Engineering

Data pipelines often depend on a file/partition/upstream job that may appear later. Busy-looping a normal task wastes resources and hides the wait state.

Picture it this way

A sensor is a receptionist periodically checking whether a delivery has arrived instead of making the entire factory line spin in a tight loop.

Start from zero

- A sensor represents waiting for a condition such as file existence, upstream partition or external task.
- Configure poke/reschedule/deferrable behavior according to operator/provider so waiting does not waste a worker unnecessarily.
- Always define timeout/failure behavior; an expected file may never arrive.
- The condition should identify the correct data interval/object, not "any file exists".
- Waiting does not validate the file/content; downstream validation still required.
- For tiny local training you can also model waiting conceptually without installing every provider package.

Teacher demonstration

```
# Conceptual condition
expected_path = f"/landing/orders/{logical_date}.csv"
# sensor waits for THIS interval-specific object
# timeout -> task failure/alert, not infinite wait
```

Read the example like English

- The path is derived from logical data interval.
- A timeout converts an absent dependency into visible failure.
- After presence, validation still checks schema/content.

Expected check

good sensor condition	bad condition
exact interval file/partition	sleep(3600) blindly
external job state with run key	any marker file
bounded timeout	infinite loop

Common beginner traps

- Using sensor as data-quality test.
- Waiting forever.
- Checking a non-interval-specific path and consuming stale file.

Now you do a similar task

Design a sensor/wait condition for daily source file arrival with interval-specific path and timeout. Explain what happens after timeout and after file appears.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is "file exists" not enough unless you know it is the file for this run’s interval?

Answer in your own words before continuing.

DE10-17 - Logs and debugging

What you will be able to do

Debug from the specific failed task log/config/mount/network state instead of rewriting code from guesswork.

Why this matters in real Data Engineering

Containers and orchestrators add layers; disciplined narrowing prevents chasing the wrong layer.

Picture it this way

Troubleshooting is a medical diagnosis: identify patient/run/task, read symptoms/log, test the smallest responsible layer, then repair.

Start from zero

- First identify exact DAG run/task instance/try and read its task log.
- For containerized tasks also inspect compose/container state, mount paths, service hostname/ports and secret path.
- Classify failure: parse/import, configuration, dependency/network, credential, deterministic data/code, transient infrastructure.
- Reproduce the smallest failing command locally/container shell where safe.
- Do not clear/rerun until you know whether repeating side effects is safe.
- After fix, rerun only necessary scope and validate data outputs/reconciliation.

Teacher demonstration

```
# Docker/Compose triage
docker compose config
docker compose ps
docker compose logs --tail=200 pipeline
# Airflow triage
# exact DAG run -> failed task -> try log -> exception -> classify
```

Read the example like English

- config catches Compose interpolation/schema issues.
- ps shows service health/state.
- service log shows application-level failure.
- Airflow task log provides orchestration context/try.

Expected check

symptom	first evidence
DAG missing	import/parse errors
task failed	task log
DB refused	service health/DNS/port/log
file missing	mount + interval path
duplicate target after retry	idempotency/load logic

Common beginner traps

- Restarting everything before reading evidence.
- Changing ports for a file-path error.
- Clearing task repeatedly and causing duplicate writes.

Now you do a similar task

Use a supplied/offline incident scenario or create a controlled missing-file error. Write a five-step diagnosis with exact evidence before/after repair.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why should you inspect the failed task log before clearing the task?

Answer in your own words before continuing.

DE10-18 - Catchup, backfills and reprocessing

What you will be able to do

Backfill historical intervals without using current-time paths, double-loading durable data or overwhelming dependencies.

Why this matters in real Data Engineering

Backfills intentionally create/reprocess many historical runs. Every assumption about interval identity, idempotency and capacity becomes visible.

Picture it this way

A backfill is reopening old daily ledgers one by one; each ledger must write to its own historical page, not today's page.

Start from zero

- Backfill means intentionally processing historical logical intervals.
- DAG/task code must derive input/output from logical interval, not wall-clock now.
- Target writes must be idempotent/partition-aware; rerunning one interval should replace/upsert its own data only.
- catchup/scheduler behavior and explicit backfill commands/UI should be understood before creating a large run set.
- Limit concurrency/rate when source/target cannot handle many historical runs.
- Do not advance unrelated global state incorrectly; each interval/run needs traceable lineage.
- Validate aggregate controls after backfill and compare affected windows.

Teacher demonstration

```
# Wrong
output = f"sales_{date.today()}.parquet"
# Right idea
output = f"sales_{data_interval_start.date()}.parquet"
```

Read the example like English

- Historical retry uses the original interval date under the right pattern.
- The wrong pattern overwrites/duplicates today's output during every old run.
- Idempotent interval writes make repeated backfill safer.

Expected check

backfill risk	control
wrong partition	logical interval paths
duplicates	upsert/partition replace
source overload	bounded concurrency/rate
state corruption	interval-specific/run state
silent wrong totals	post-backfill reconciliation

Common beginner traps

- Using current date/time for source/target partition.
- Launching hundreds of intervals without capacity plan.
- Assuming catchup=false means historical reruns are impossible.

Now you do a similar task

Write a backfill plan for Aug 1-7: run identity, input/output partition, retry policy, concurrency and validation. Explain how you safely rerun Aug 3 twice.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What property must the Aug 3 load have so a second backfill of Aug 3 does not double data?

Answer in your own words before continuing.

DE10-19 - Integration and production-style orchestration

What you will be able to do

Integrate Docker packaging, Compose dependencies and Airflow orchestration into a reproducible pipeline with safe retries/backfills/secrets and operational evidence.

Why this matters in real Data Engineering

The integration practical tests whether you can distinguish runtime, network/storage/config and orchestration correctness at the same time.

Picture it this way

You are shipping both the factory machinery (container images/services) and the control tower schedule (Airflow), then proving the product data is correct after failures/replays.

Start from zero

- Validate Dockerfile/Compose statically before runtime.
- Containerize the pipeline without embedding secrets; persist DB/output intentionally.
- Use service DNS/healthchecks for Compose dependencies.
- Define Airflow tasks around meaningful retry boundaries and logical intervals.
- Use idempotent load/reconciliation so task retry and backfill are safe.
- Capture logs/run/task evidence and final data controls.
- A green Airflow DAG plus running containers is not enough; validate target rows/rejects/control totals and clean replay.

Teacher demonstration

```
# Production-style proof
compose_config_valid == True
no_secret_in_image_or_git == True
services_healthy_or_failure_explained == True
dag_parses == True
failed_task_retry_is_idempotent == True
backfill_writes_correct_interval == True
source_rows == valid_rows + rejected_rows
replay_does_not_double_target == True
```

Read the example like English

- These invariants span packaging, orchestration and data correctness.
- The supplied project also includes offline validators for environments where Docker/Airflow cannot run.
- Live Docker/Airflow smoke evidence must be collected on the learner PC before marking the runtime portion complete.

Expected check

layer	proof
Docker build/config	image/compose validation
runtime	services/health/logs
Airflow	DAG parse + run/task states
data	row/key/measure reconciliation

layer	proof
replay/backfill	idempotency + correct interval

Common beginner traps

- Marking module complete using offline validator only when live Docker is available but never tested.
- Calling Airflow green = data correct.
- Testing secrets with real production credentials.

Now you do a similar task

Complete the helpdesk module practical from a fresh assessment asset. Save compose validation, task/run reasoning and data controls. If Docker/Airflow run on your PC, perform the live smoke test and one safe rerun/backfill.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What evidence comes from Airflow, and what separate evidence must come from the data target?

Answer in your own words before continuing.